Fast and Differentiable Sorting and Ranking: Parallelization

...

Nirmal Patel

Code: https://github.com/nirmal129/Parallel-torchsort

Theory and History

- First introduced in **Fast Differentiable Sorting and Ranking** paper by Blondel et al. in
- Defines sorting and ranking as optimization over Permutahedron (convex set whose vertices correspond to a permutation)
- Uses permutahedron-projections and introduces soft sorting and ranking that are differentiable a.e.
- Has O(nlog(n)) complexity and exact formulation compared to $O(n^2)$ complexity of Optimal-Transport based relaxation or approximate formulation relying on constraint-optimization
- Is integrated in **SoftJAX & SoftTorch**, 2026

Soft Sort: Forward Pass

- **Inputs:** Input tensor $\mathbf{x}$, regularization strength ε, and regularization type (default L2)
- **Step 1 (Target Weights):** Get $\mathbf{w}$ as $w_i = (n - i + 1) / \varepsilon$ for $i = 1$ to n
- **Step 2 (Standard Sort):** $\mathbf{s}, \boldsymbol{\pi} = \text{sort}(-\mathbf{x})$
- **Step 3 (Isotonic Regression):** $\boldsymbol{\theta} = \text{argmin}_{\mathbf{z}} \|\mathbf{z} - (\mathbf{w} - \mathbf{s})\|^2$ s.t. $(z_1 \geq z_2 \geq \ldots \geq z_n)$for L2
- **Step 4 (Mapping to Output):** $\mathbf{y} = -(\mathbf{w} - \boldsymbol{\theta})$
- **Outputs:** The differentiable sorted tensor $\mathbf{y}$

Soft Sort: Backward Pass

- **Inputs:** Output Gradient $\nabla \mathbf{y}$, saved tensors from the forward pass ($\mathbf{s}$, $\boldsymbol{\theta}$, $\boldsymbol{\pi}$).
- **Step 1 (Isotonic Gradient):** $\nabla \mathbf{s}$ = Isotonic_Backward_L2($\mathbf{s}$, $\boldsymbol{\theta}$, $\nabla \mathbf{y}$)
- **Step 2 (Gradient Scattering):** $\nabla \mathbf{x}$ = gather($\nabla \mathbf{s}$, $\boldsymbol{\pi}^{-1}$)
- **Outputs:** The gradient w.r.t. the input $\nabla \mathbf{x}$

Isotonic Regression

- Get **z** that minimize $\|\mathbf{z} - \mathbf{y}\|^2$ for a given **y** such that $z_1 \geq z_2 \geq \ldots \geq z_n$
- **Pool Adjacent Violator (PAV)** solves this exactly in O(n)

```
# Initialize: Every element is a 'block'
Blocks = [ (y_1), (y_2), ..., (y_n) ]

For each Block:
    # While current block > previous block (Violation!)
    While Current.Value > Previous.Value:
        # 1. POOL: Merge them into one block
        New_Block = Merge(Previous, Current)
        # 2. AVERAGE: Update value to arithmetic mean
        New_Block.Value = Sum(elements) / Count(elements)
        # 3. BACKTRACK: Check if New_Block violates the one before it
        Current = New_Block
```

Parallelizing PAV: Divide-&-Conquer

- While **PAV** is inherently sequential as it needs to backtrack in case of violation, a **Divide-&-Conquer** strategy can be implemented.

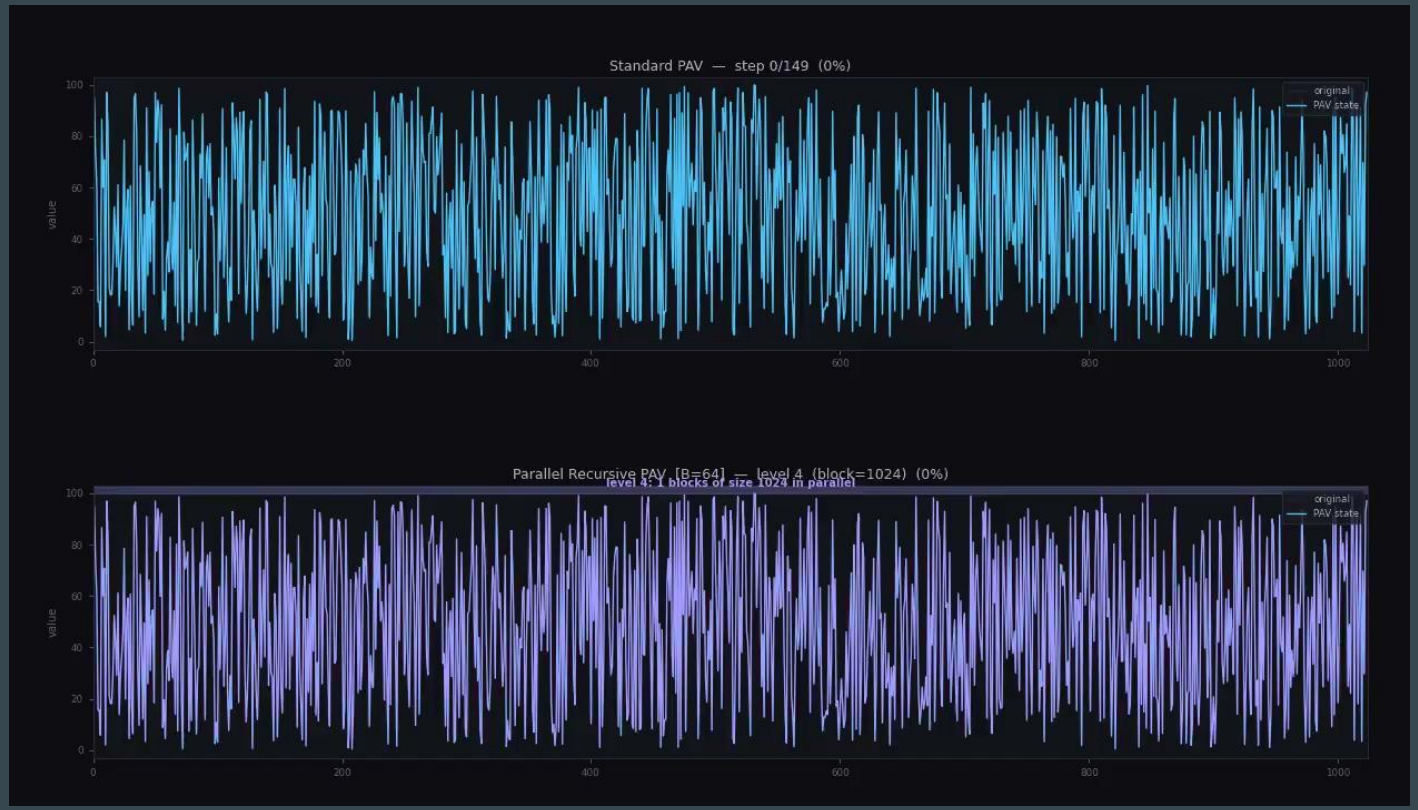
```
def Parallel_PAV(y):
    if len(y) == 1:
        return [Block(value=y[0], size=1)]
    # 1. DIVIDE: Split and solve in parallel
    L_blocks = Parallel_PAV(left_half)
    R_blocks = Parallel_PAV(right_half)
    # 2. CONQUER: Merge at the boundary
    return Merge(L_blocks, R_blocks)
```

Parallelizing PAV: Merge

- As left and right slices are **PAV**ed, the violation is at the junction.
- Apply sequential **PAV** (on the blocks rather than elements) starting from the junction.

```
def Merge(Left_Blocks, Right_Blocks):
    # Check the junction for a violation (Left < Right)
    while Left_Blocks.last < Right_Blocks.first:
        # 1. POOL: Pop the two violating blocks
        B1, B2 = Left_Blocks.pop(), Right_Blocks.pop_first()
        # 2. AVERAGE: Merge into a single weighted mean
        merged = WeightedMean(B1, B2)
        # 3. RE-INSERT: Push back to check for new violations
        Left_Blocks.push(merged)
    return concatenate(Left_Blocks, Right_Blocks)
```

Parallelizing PAV: Animation



Parallelizing PAV: PRAM Analysis

- Firstly, p processors will perform sequential **PAV** on p chunks parallely.
- Then, p/2 processors will perform the merge step parallely.
- The tree reduction continues till only one final merge step is remaining, giving depth of log(p).
- Work and Time complexities depends on the complexity of merges at each level.
- It is not work optimal for any case.
- Benefit of using **D&C PAV** is reciprocal to how close **y** is to **PAV** solution.

Parallelizing PAV: PRAM Analysis

- **Best Case (non-decreasing y)**
  - Base case and each merge produce a mega block
  - Maximum Single Merge Complexity at any level: $O(1)$
  - Base Case PAV: Work = $O(n)$, Time = $O(n/p)$
  - Total Work Complexity: $O(n + 2p - 1)$
  - Total Time Complexity: $O(n/p + \log(p))$
  - Speedup: $S(n, p) = O(np / (n + p\log(p)))$
  - Efficiency: $E(n, p) = O(n / (n + p\log(p)))$

Parallelizing PAV: PRAM Analysis

- **Worst Case (non-increasing y)**
  - Base case and each merge preserve all elements
  - Maximum Single Merge Complexity at any level: $O(2^L n/p)$
  - Base Case PAV: Work = $O(n)$, Time = $O(n/p)$
  - Total Work Complexity: $O(n + n\log(p))$
  - Total Time Complexity: $O(n/p * (1 + 2 + 4 + ... + p)) = O(n)$
  - Speedup: $S(n, p) = O(1)$
  - Efficiency: $E(n, p) = O(1/p)$

Parallelizing PAV: PRAM Analysis

- **Average Case (at each merge level, half of blocks are merged)**
  - Base case and each merge generate #blocks = #elements/2
  - Maximum Single Merge Complexity at any level: $O(n/p)$
  - Base Case PAV: Work = $O(n)$, Time = $O(n/p)$
  - Total Work Complexity: $O(n(3 - 1/p))$
  - Total Time Complexity: $O(n(1 + \log(p))/p)$
  - Speedup: $S(n, p) = O(p/(1 + \log(p)))$
  - Efficiency: $E(n, p) = O(1/(1 + \log(p)))$

Isotonic Regression Backward Pass: Highly Parallelizable

```
# 1. PARTITION: Identify the flat regions (blocks) in the
solution
Blocks = Identify_Constant_Segments(s_pav)
# 2. GATHER & SCATTER: Process each block independently
For each Block:
    # Gather sum of incoming gradients for this range
    Grad_Sum = sum(nabla_y[Block_Start : Block_End])
    # Calculate average gradient
    Grad_Mean = Grad_Sum / Block_Size
    # Scatter average back to all elements in the block
    nabla_s[Block_Start : Block_End] = Grad_Mean
```

Find start of each **PAV** blocks

Take mean of the corresponding blocks of $\nabla \mathbf{y}$

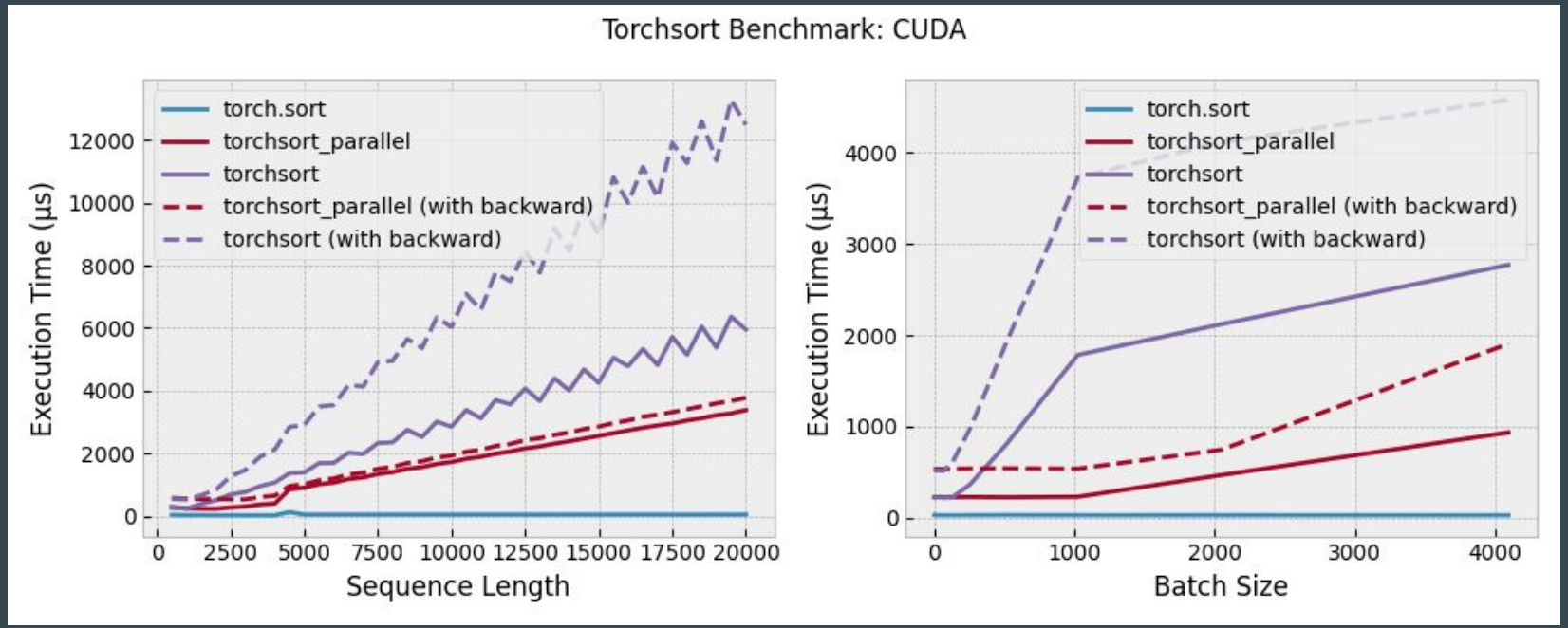
Scatter the means to the corresponding blocks of $\nabla \mathbf{s}$

Implementation

- Forked **torchsort** repo of **Teddy Koker**
- The implementation is batch-wise parallel and sequence-wise serial
- Wrote two CUDA kernels: forward and backward, that are batch-wise and sequence-wise parallel using the current implementation as a reference
- Has bugs, couple of unit-tests fail with single element mismatch, but works so far

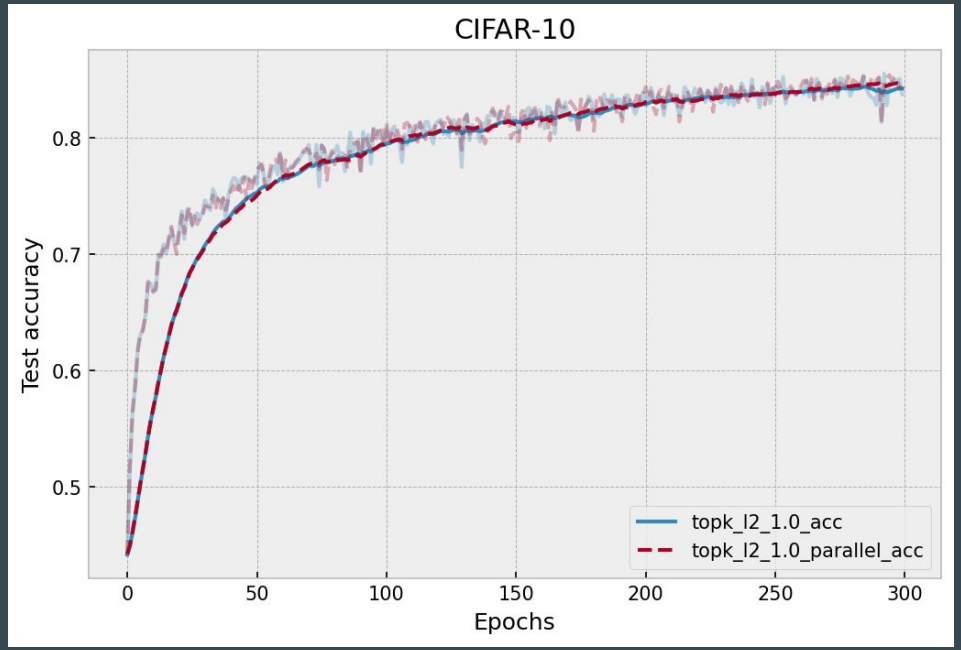
Results: Wall-Clock Time

(Left) Varying Sequence Lengths, Batch Size 4, (Right) Varying Batch Sizes, Sequence Length 512



Results: Correctness Check

Apart from unit-tests, identical models with topK loss, which checks if the true label is in top K classes, were trained on **cifar10** to verify the stability of the parallelized kernels



Conclusion

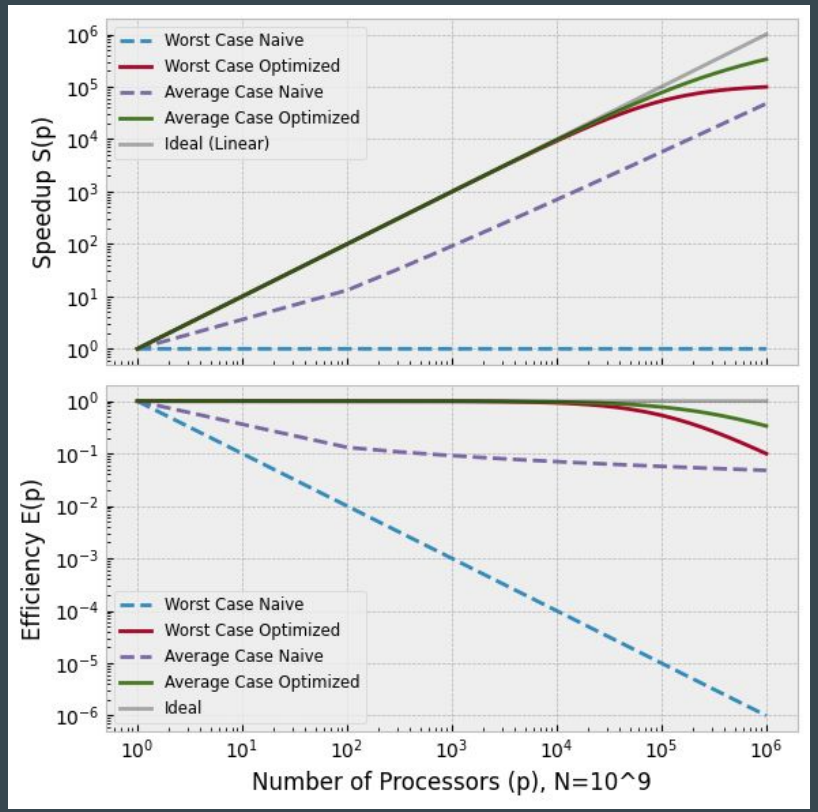
- While **PAV** is parallelizable to an extent, my implementation is still naive.
- **PAV** is equivalent to **Y** = scan(**y**), finding the upper convex hull of (i, Y_i), and then scattering the slopes [4]. So, CUDA kernels of convex hull finding, like CudaHull or CudaChain, can be modified to directly support **PAV**.

Frontier (I believe, Don't quote me)

Considering that slopes of left and right convex hulls are individually sorted and both are x-coord sorted and separated, I believe merge with complexity O(log(L)log(R)) can be conjured. This gives following time complexities:

- Best Case: O(n/p + log(p))
- Worst Case: O(n/p+log(2p)(log(n)log(n/p)+log(p)log(2p^2)/6))
- Average Case: O(n/p + log(p)(log(n/p))^2)

(Naive in plot refers to naive merge while optimized refers to O(log(L)log(R)) merge)



References

[1] **Fast Differentiable Sorting and Ranking:** https://arxiv.org/abs/2002.08871

[2] **SoftJAX & SoftTorch:** https://arxiv.org/abs/2603.08824

[3] **torchsort:** https://github.com/teddykoker/torchsort

[4] **PAV and the ROC Convex Hull:**
https://link.springer.com/article/10.1007/s10994-007-5011-0

Thank You